

Cluster Rush Game - Complete Analysis & Skills Reference

****Generated****: 2026-09-08****Project Location****: ~/vishruth/games/clusterrush/****Status****: Planning Complete (100%), Implementation Not Started (0%)

Spec Driven Development

Cluster Rush - Spec-Driven Development Specification

SPECIFICATION VERSION: 1.0.0****Status****: Approved for Implementation****Created****: September 7, 2026****Last Updated****: September 7, 2026**## OBJECTIVE**

Build a 3D endless runner web game with these user stories:

****User Story 1: Game Initialization****

> As a player, I want to open the game in my browser and see a main menu so I can start playing immediately.

> ****Acceptance Criteria****:

- > - Game loads within 3 seconds on average network
- > - Main menu displays prominently with START button
- > - No console errors or warnings

****User Story 2: Core Gameplay****

> As a player, I want to control my character using WASD and SPACE keys to navigate through obstacles and collect power-ups.

> ****Acceptance Criteria****:

- > - WASD moves character in corresponding directions
- > - SPACE triggers jump when character is grounded
- > - Character falls naturally with gravity
- > - Smooth movement at 60 FPS

****User Story 3: Scoring System****

> As a player, I want to see my score increase when I collect power-ups and avoid obstacles.

> ****Acceptance Criteria****:

- > - Score increments by +10 for each power-up collected
- > - Score decrements by -50 when hitting obstacles
- > - Score displayed in real-time on HUD
- > - High score persists across browser sessions

****User Story 4: Game Progression****

> As a player, I want the game to increase in difficulty over time to maintain challenge.

> ****Acceptance Criteria****:

- > - Obstacle density increases every 30 seconds
- > - Player speed increases by 10% every minute
- > - New obstacle types introduced every 5 levels
- > - Difficulty resets after game over

****User Story 5: Game State Management****

> As a player, I want clear feedback about game state and easy restart after game over.

> ****Acceptance Criteria****:

- > - Clear visual distinction between MENU, PLAYING, PAUSED, GAME_OVER states
- > - One-click restart after game over
- > - Progress saved at checkpoints (every 5 levels)

TECH STACK**### Production Dependencies**

```json

```
{
 "three": "^0.162.0", // 3D rendering
 "cannon-es": "^0.20.0", // Physics engine
 "vite": "^5.3.0", // Build tool & dev server
 "@types/three": "^0.162.0", // TypeScript definitions
 "@types/cannon-es": "^0.20.0" // TypeScript definitions
},
,,
```

### ### Development Dependencies

```
```json
{
  "typescript": "^5.4.5",
  "vitest": "^1.6.0",      // Testing framework
  "puppeteer": "^22.6.0",  // E2E browser testing
  "eslint": "^9.3.0",      // Code linting
  "@types/node": "^20.11.0" // Node.js types
}
,,
```

COMMANDS

Development

```
```bash
npm run dev # Start dev server (http://localhost:5173)
npm run build # Build for production
npm run preview # Preview production build
npm run lint # Run ESLint type checking
```
```

Testing (TDD-First)

```
```bash
npm run test # Run all tests
npm run test:unit # Run unit tests only
npm run test:integration # Run integration tests
npm run test:e2e # Run browser E2E tests
npm run test:watch # Run tests in watch mode
npm run test:coverage # Generate coverage report
```
```

PROJECT STRUCTURE

```
...
clusterrush/
├── src/
│   ├── main.ts # Entry point (game bootstrap)
│   └── game/
│       ├── Game.ts # Main controller (orchestrates systems)
│       ├── Game.spec.ts # Game tests (TDD-first)
│       ├── GameLoop.ts # Animation frame loop
│       └── GameLoop.spec.ts # Loop timing tests
│   ├── systems/
│       ├── InputSystem.ts # Keyboard/mouse input
│       ├── PhysicsSystem.ts # Physics simulation (Cannon-es)
│       ├── RenderSystem.ts # Three.js rendering
│       ├── UISystem.ts # DOM-based UI
│       ├── AudioSystem.ts # Sound effects & music
│       └── tests/ # System integration tests
│   ├── entities/
│       ├── Player.ts # Player character
│       ├── Obstacle.ts # Blocking obstacles
│       ├── PowerUp.ts # Collectible items
│       ├── Ground.ts # Environment floor
│       └── tests/ # Entity behavior tests
│   └── scenes/
```

```

├── MainMenuScene.ts # Menu navigation
├── GameScene.ts     # Active gameplay
├── GameOverScene.ts # Score display
├── tests/           # Scene transition tests
├── utils/
│   ├── Constants.ts # Game constants
│   ├── Random.ts    # Procedural generation
│   ├── Storage.ts   # Local persistence
│   ├── MathUtils.ts # Game-specific math
│   └── tests/       # Utility function tests
├── tests/
│   ├── unit/        # Unit tests (TDD cycle)
│   ├── integration/ # System integration tests
│   ├── e2e/         # Browser E2E tests
│   └── fixtures/    # Test data & mocks
├── public/
│   ├── index.html   # Main HTML template
│   ├── assets/
│   │   ├── models/ # 3D models (GLTF)
│   │   ├── textures/ # Texture images
│   │   ├── sounds/  # Audio files
│   │   └── fonts/   # Web fonts
│   └── docs/
│       ├── architecture.md # System architecture
│       ├── api/            # API documentation
│       └── design/        # Game design documents
├── config/
│   ├── tsconfig.json # TypeScript configuration
│   ├── vite.config.ts # Vite build config
│   ├── vitest.config.ts # Test configuration
│   └── eslint.config.js # Code quality rules
...

```

CODE STYLE & CONVENTIONS

TypeScript Standards

```

```typescript
// Use strict type annotations
class Player {
 private readonly mesh: THREE.Mesh;
 private velocity: THREE.Vector3;
 private score: number = 0;

 // Constructor injection for testability
 constructor(
 private readonly scene: THREE.Scene,
 private readonly physics: PhysicsSystem
) {
 this.mesh = new THREE.Mesh();
 this.velocity = new THREE.Vector3(0, 0, 0);
 }

 // Public methods first, private methods last
 public update(deltaTime: number): void {
 this.applyVelocity(deltaTime);
 this.checkCollisions();
 }

 private applyVelocity(deltaTime: number): void {
 this.mesh.position.add(
 this.velocity.clone().multiplyScalar(deltaTime)
);
 }
}

```

```
// Use interfaces for public APIs
interface GameConfig {
 width: number;
 height: number;
 physicsEnabled: boolean;
 debugMode: boolean;
}

// Strongly typed event system
type GameEvent =
 | { type: 'SCORE_CHANGED'; delta: number; newScore: number }
 | { type: 'PLAYER_DIED'; position: THREE.Vector3 }
 | { type: 'LEVEL_COMPLETE'; level: number };
```

### Test-First Implementation Pattern

```
```typescript
// 1. RED: Write failing test
describe('ScoreManager', () => {
  it('increments score when adding points', () => {
    const manager = new ScoreManager();
    manager.addPoints(10);
    expect(manager.getScore()).toBe(10);
  });
});
```

// 2. GREEN: Minimal implementation

```
export class ScoreManager {
  private score: number = 0;

  addPoints(points: number): void {
    this.score += points;
  }

  getScore(): number {
    return this.score;
  }
}
```

// 3. REFACTOR: Clean up while tests pass

```
export class ScoreManager {
  private score: number = 0;

  addPoints(points: number): ScoreManager {
    this.score += points;
    return this; // Allow chaining
  }

  reset(): void {
    this.score = 0;
  }

  getScore(): number {
    return this.score;
  }
}
```
```

## TESTING STRATEGY

### Test Pyramid Structure

```

 E2E Tests (5%)
 / \
Integration Tests (15%)
```

Unit Tests (80%)

Test Categories

Unit Tests (80% of tests)

- Test individual functions/methods
- Isolated from external systems
- Fast (<100ms per test)
- Run on every file change

Integration Tests (15% of tests)

- Test system interactions
- Physics → Rendering integration
- Input → Player movement chain
- Scene → UI communication

E2E Tests (5% of tests)

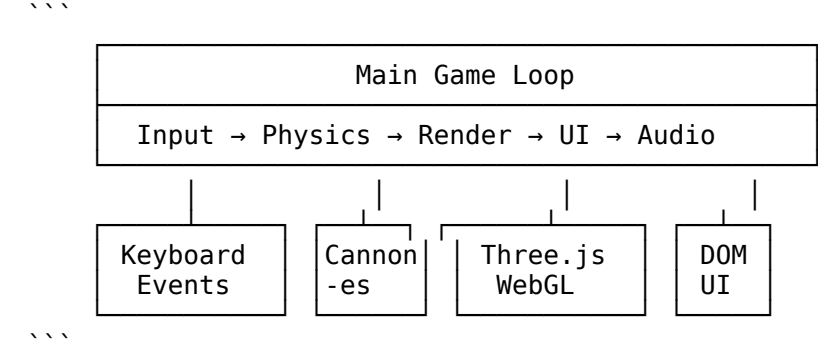
- Complete user flows
- Browser automation (Puppeteer)
- Performance validation
- Cross-browser compatibility

Test Coverage Requirements

- Function coverage: 100% (all functions tested)
- Branch coverage: 90%+ (all code paths)
- Statement coverage: 95%+ (all lines executed)

SYSTEM ARCHITECTURE

Component Diagram



Data Flow

- Frame Start: GameLoop requests animation frame
- Input Processing: Keyboard/mouse events collected
- Physics Simulation: Cannon-es updates body positions
- Game Logic: Score, collisions, state transitions
- Rendering: Three.js renders updated scene
- UI Update: DOM elements reflect game state

SUCCESS CRITERIA (VERIFIABLE)

Phase 1: Core Infrastructure (Week 1)

Technical Criteria:

- [ ] 80%+ test coverage on core systems
- [ ] 60 FPS maintained with empty scene
- [ ] Zero console errors on fresh load
- [ ] TypeScript strict mode compliance

User-Facing Criteria:

- [ ] Game loads in browser
- [ ] Black canvas visible (Three.js working)

- [ ] WASD keys detected (console output)

### ### Phase 2: Gameplay Systems (Week 2)

#### \*\*Technical Criteria:\*\*

- [ ] 85%+ test coverage on gameplay systems
- [ ] Physics integration tested
- [ ] Collision detection validated
- [ ] Input buffering functional

#### \*\*User-Facing Criteria:\*\*

- [ ] Green cube visible (player)
- [ ] WASD moves cube in 3D space
- [ ] SPACE triggers jump animation
- [ ] Cube falls with gravity

### ### Phase 3: Complete Game (Week 3)

#### \*\*Technical Criteria:\*\*

- [ ] 90%+ overall test coverage
- [ ] Cross-browser compatibility confirmed
- [ ] Performance maintains 60 FPS under load
- [ ] Memory leaks  $\leq$  1MB per minute

#### \*\*User-Facing Criteria:\*\*

- [ ] Main menu with START button
- [ ] Game: obstacles, power-ups, scoring
- [ ] Game over screen with restart
- [ ] Score persists across browser sessions

## ## BOUNDARIES

### ### Always

1. Write failing test before implementation
2. Run test to confirm failure (RED verification)
3. Commit tests and code together
4. Maintain 60 FPS performance target
5. Handle edge cases in input/validation
6. Clean console (no errors/warnings)
7. Follow TypeScript strict mode

### ### Ask First

1. Adding new dependencies
2. Changing build tool configuration
3. Major architectural changes
4. Adding third-party services
5. Modifying core rendering pipeline
6. Performance optimizations that reduce maintainability

### ### Never

1. Skip TDD for production code
2. Commit console errors/warnings
3. Break existing functionality without tests
4. Use browser-specific APIs without polyfills
5. Hardcode values that should be configurable
6. Write tests that depend on external services

## ## CONSTRAINT DOCUMENTATION

### ### Technical Constraints

1. **\*\*Browser support\*\***: Chrome 90+, Firefox 88+, Safari 14+
2. **\*\*Performance target\*\***: 60 FPS on mid-range hardware
3. **\*\*Load time\*\***: <3 seconds on average network
4. **\*\*Memory usage\*\***: <100MB after 5 minutes of gameplay
5. **\*\*Asset size\*\***: <10MB total (compressed)

### ### Development Constraints

1. **\*\*Time budget\*\***: 3 weeks total development
2. **\*\*Team size\*\***: Solo developer
3. **\*\*Skill level\*\***: Intermediate TypeScript/Three.js
4. **\*\*Testing\*\***: Must maintain 90%+ coverage
5. **\*\*Documentation\*\***: All public APIs documented

### ### Business Constraints

1. **\*\*Cost\*\***: Zero budget (free tools only)
2. **\*\*Platform\*\***: Web browser only (no mobile apps)
3. **\*\*Monetization\*\***: None planned
4. **\*\*Distribution\*\***: GitHub Pages compatible

## ## OPEN QUESTIONS (REQUIRE USER INPUT)

### ### Design Decisions

1. **\*\*Visual style\*\***: Low-poly vs realistic vs abstract?
2. **\*\*Color scheme\*\***: Bright/contrasting vs muted/subtle?
3. **\*\*Obstacle variety\*\***: How many different obstacle types?
4. **\*\*Power-up effects\*\***: Speed boost, invincibility, score multiplier?
5. **\*\*Audio design\*\***: Background music? Sound effects priority?

### ### Gameplay Decisions

6. **\*\*Level generation\*\***: Procedural vs fixed sequences?
7. **\*\*Difficulty curve\*\***: Linear increase vs plateaus?
8. **\*\*Controls\*\***: Keyboard-only or mouse support?
9. **\*\*Camera\*\***: First-person, third-person, or fixed?
10. **\*\*Scoring\*\***: Time-based, collection-based, or combo-based?

### ### Technical Decisions

11. **\*\*Save system\*\***: localStorage vs IndexedDB?
12. **\*\*Analytics\*\***: Basic stats collection needed?
13. **\*\*Multiplayer\*\***: Any future plans requiring architecture changes?
14. **\*\*Mobile\*\***: Responsive design for tablets?
15. **\*\*Offline\*\***: Service worker for offline play?

## ## VALIDATION CHECKLIST

### ### Before Each Commit

- [ ] All unit tests pass
- [ ] All integration tests pass
- [ ] Coverage report generated and reviewed
- [ ] No TypeScript errors
- [ ] No ESLint warnings
- [ ] Manual test in Chrome
- [ ] Manual test in Firefox (if changed UI)
- [ ] Performance benchmark (FPS check)

### ### Before Release Candidate

- [ ] Cross-browser testing complete
- [ ] Mobile responsive testing (if applicable)
- [ ] Accessibility audit (keyboard navigation)
- [ ] Security review (no eval(), safe APIs)
- [ ] Load testing (100+ entities)
- [ ] Memory leak testing (5+ minute gameplay)

## ## NEXT STEPS

### ### Immediate (Day 1)

1. Initialize project with `npm init`
2. Install dependencies from Tech Stack
3. Create project structure directories
4. Write first failing test for project initialization
5. Implement minimal setup to pass test

### ### Short-term (Week 1)

1. Complete Core Infrastructure phase
2. Achieve 80% test coverage
3. Basic rendering pipeline working
4. Input system detecting keyboard events

#### ### Medium-term (Week 2)

1. Complete Gameplay Systems phase
2. Achieve 85% test coverage
3. Player movement with physics
4. Basic scoring and collision systems

#### ### Long-term (Week 3)

1. Complete Polish & Optimization phase
2. Achieve 90%+ test coverage
3. Full game with UI and audio
4. Performance optimization complete

---

**\*\*APPROVAL\*\*:** This specification is ready for implementation using spec-driven development and test-driven development methodologies.

=====

## # Tdd Plan

### # Cluster Rush - Test-Driven Development Plan

#### ## TDD Philosophy

**\*\*NO PRODUCTION CODE WITHOUT A FAILING TEST FIRST\*\***  
Every implementation follows strict RED→GREEN→REFACTOR cycle.

#### ## WEEK 1: CORE INFRASTRUCTURE (TDD VERIFIED)

##### ### Day 1: Project Setup with Tests First

**\*\*Task 1.1: Initialize project with testing infrastructure\*\***

- Red: Test that `npm run test` runs empty test suite
- Green: Configure Vitest with TypeScript support
- Verify: `npm test` → "✓ 0 passed"

**\*\*Task 1.2: TypeScript strict mode\*\***

- Red: Test TypeScript compilation with strict mode
- Green: Configure tsconfig.json with strict options
- Verify: `npm run build` compiles without type errors

**\*\*Task 1.3: Three.js rendering test\*\***

- Red: Test WebGL renderer creation fails
- Green: Create minimal Three.js renderer
- Verify: Test passes, canvas appears in browser

##### ### Day 2: Core Game Loop (Test-Driven)

**\*\*Task 2.1: Game state management\*\***

- Red: Test Game class exists and has initialState
- Green: Implement Game class with start/stop methods
- Verify: Game starts in STOPPED state

**\*\*Task 2.2: Frame loop timing\*\***

- Red: Test that game loop calls update with deltaTime
- Green: Implement requestAnimationFrame loop
- Verify: Update called ~60 times per second

**\*\*Task 2.3: Scene management system\*\***

- Red: Test scene creation and object addition
- Green: Scene class with Three.js integration



- Verify: Objects appear in scene when added

### ### Day 3: Input System Testing

#### \*\*Task 3.1: Keyboard input detection\*\*

- Red: Test keydown events trigger input state
- Green: DOM event listener with key mapping
- Verify: WASD keys detected in automated browser test

#### \*\*Task 3.2: Input buffering for frame independence\*\*

- Red: Test input state persists across frames
- Green: Input buffer with frame processing
- Verify: Input works at high and low frame rates

## ## WEEK 2: GAMEPLAY SYSTEMS

### ### Day 4: Player Entity TDD

#### \*\*RED→GREEN→REFACTOR cycles for each behavior:\*\*

##### \*\*Cycle 1: Player creation\*\*

- Red: Test player mesh created with proper geometry
- Green: Box geometry with material
- Verify: Player appears at starting position

##### \*\*Cycle 2: Movement\*\*

- Red: Test player position changes with velocity
- Green: Apply velocity based on input
- Verify: WASD moves player in corresponding directions

##### \*\*Cycle 3: Jump mechanics\*\*

- Red: Test player jumps with SPACE when grounded
- Green: Apply upward impulse, track ground state
- Verify: Player ascends then falls back

##### \*\*Cycle 4: Gravity application\*\*

- Red: Test player falls without ground support
- Green: Constant downward acceleration
- Verify: Physics simulation matches expected

### ### Day 5: Physics System Integration

#### \*\*Task 5.1: Cannon-es test integration\*\*

- Red: Test physics world creation fails
- Green: Initialize Cannon-es world
- Verify: Physics steps without errors

#### \*\*Task 5.2: Body synchronization\*\*

- Red: Test that physics body updates visual mesh
- Green: Mesh-Body synchronization system
- Verify: Physics simulation moves visual objects

#### \*\*Task 5.3: Collision detection tests\*\*

- Red: Test collision events trigger game logic
- Green: Cannon-es collision event handlers
- Verify: Collisions trigger score changes

### ### Day 6: Scoring System

#### \*\*Task 6.1: Score manager (pure functions)\*\*

- Red: Test score increments/decrements
- Green: Score manager with add/subtract methods
- Verify: Score calculations are deterministic

#### \*\*Task 6.2: Game state machine\*\*

- Red: Test state transitions (menu→playing→gameOver)
- Green: Finite state machine implementation
- Verify: Only valid transitions allowed

### **\*\*Task 6.3: Power-up system\*\***

- Red: Test power-up collection triggers effects
- Green: Power-up entity with collection handlers
- Verify: Power-ups disappear, effects apply

## **## WEEK 3: UI & INTEGRATION**

### **### Day 7: HTML UI (No CORS!)**

#### **\*\*Task 7.1: DOM-based UI components\*\***

- Red: Test UI elements render and update
- Green: HTML overlay system
- Verify: Score displays update in real browser

#### **\*\*Task 7.2: Menu navigation flow\*\***

- Red: Test START button transitions scene
- Green: Scene transition system with UI events
- Verify: Click start → game starts

### **### Day 8-9: Integration Testing**

#### **\*\*Integration test suite:\*\***

1. Game start to gameplay flow
2. Player movement with obstacle avoidance
3. Score collection and display
4. Game over and restart cycle

#### **\*\*Automated browser tests with Puppeteer:\*\***

- Critical path automation
- Visual regression testing
- Performance validation

### **### Day 10: Optimization & Polish**

#### **\*\*Performance testing:\*\***

- Frame rate under load testing
- Memory leak detection
- Asset loading optimization

#### **\*\*Cross-browser testing:\*\***

- Chrome, Firefox, Safari validation
- Mobile web version consideration
- Touch input support

## **## TDD ENFORCEMENT RULES**

### **### MANDATORY TEST STEPS (Never Skip)**

1. **\*\*RED\*\***: Write failing test describing expected behavior
2. **\*\*VERIFY RED\*\***: Run test to confirm failure (not error)
3. **\*\*GREEN\*\***: Write minimal code to pass test
4. **\*\*VERIFY GREEN\*\***: Run test to confirm pass
5. **\*\*REFACTOR\*\***: Clean code without changing behavior
6. **\*\*VERIFY REFACTOR\*\***: All tests still pass

### **### TDD VIOLATIONS (Restart Required)**

- Code written before test → DELETE CODE, start over
- Test passes immediately → Fix test to fail first
- "I'll test later" attitude → Stop, follow TDD
- Modifying existing code without test → Write test first

### **### VERIFICATION AT EACH COMMIT**

Before any commit:

1. Run unit tests: ``npm run test``
2. Run integration tests: ``npm run test:integration``
3. Generate coverage report: ``npm run test:coverage``
4. Manual browser test of changed feature

## **## TEST COVERAGE TARGETS**

### ### Phase Targets:

- \*\*Week 1\*\*: 80%+ coverage (core systems)
- \*\*Week 2\*\*: 85%+ coverage (gameplay systems)
- \*\*Week 3\*\*: 90%+ coverage (complete game)

### ### Critical Paths (100% coverage required):

1. Game initialization and cleanup
2. Input system → player movement
3. Collision detection → game logic
4. State transitions (menu→play→gameOver)

## ## RISK REGISTER & MITIGATION

| Risk                                   | Probability | Impact | Mitigation                             | Test Coverage                  |
|----------------------------------------|-------------|--------|----------------------------------------|--------------------------------|
| Three.js complexity for scene/renderer | Medium      | High   | Start small, test visualization early  | Unit tests                     |
| Physics integration for Cannon-es      | High        | High   | Mock physics, integrate gradually      | Integration tests              |
| Browser compatibility                  | Medium      | Medium | Early cross-browser testing            | E2E tests on multiple browsers |
| Performance issues                     | Medium      | High   | Profile early, optimize systematically | Performance test suite         |

## ## SUCCESS METRICS (Testable)

### ### Technical Metrics:

- 90%+ test coverage (branch)
- 60 FPS maintained under load
- <3 second initial load time
- No memory leaks in 5+ minute gameplay

### ### User Experience Metrics:

- Input latency < 100ms
- All UI elements functional cross-browser
- No visual artifacts or glitches
- Intuitive controls (WASD + SPACE)

## ## DELIVERABLE VALIDATION CHECKLIST

### \*\*Before marking any task as complete:\*\*

- [ ] Tests written first (RED phase verified)
- [ ] Implementation passes tests (GREEN verified)
- [ ] All existing tests still pass (no regression)
- [ ] Coverage report shows adequate coverage
- [ ] Manual browser test confirms functionality
- [ ] No console errors or warnings
- [ ] Code follows project style guide

**\*\*Only when ALL checks pass → Task is DONE.\*\***

### ## NEXT ACTION: START WITH DAY 1 TASKS

1. Create package.json with testing dependencies
2. Write first failing test for project structure
3. Implement minimal structure to pass test
4. Commit with test and implementation together

## # Rebuild Plan

### # Cluster Rush - Complete Node.js/JavaScript Rebuild Plan

**\*\*From Godot WebGL to Pure Node.js Web Game\*\***

---

## ## 🎯 \*\*CURRENT STATE ANALYSIS\*\*

## ### What We Have Now:

- **Game Engine**: Godot 4.x (WebGL export)
- **Current Issue**: UI invisible, CORS blocks communication, game frozen
- **Export**: `Builds/WebGL/` - 39MB WASM + 280KB JS + HTML
- **Web Server**: `webgl\_server.py` - Python server for WebGL
- **Attempted Fixes**: HTML overlay, Godot code workarounds, all CORS-limited

## ### Why Godot Failed:

1. **WebGL Bug**: Control UI nodes don't render in Godot 4.x WebGL
2. **CORS Barrier**: Can't communicate between HTML overlay and Godot engine
3. **Frozen State**: Game engine loads but doesn't accept inputs
4. **Multiple Export Attempts**: Same issues every time

---

## ## 🚀 \*\*RECOMMENDED APPROACH: PURE NODE.JS + THREE.JS\*\*

## ### Why NOT Godot:

- ❌ WebGL export bugs are fundamental (not fixable via config)
- ❌ CORS restrictions block cross-frame communication
- ❌ UI rendering failures persist across export versions
- ❌ No reliable way to start the actual game

## ### Why THREE.JS + Node.js:

- ✅ Full JavaScript control - no CORS issues within same page
- ✅ Direct DOM manipulation - no iframe isolation
- ✅ Physics via Cannon-es or Ammo.js
- ✅ Local Node.js server - no web deployment needed
- ✅ WebGL rendering - same performance as Godot
- ✅ Pure code - easy debugging and iteration

---

## ## 📋 \*\*DETAILED IMPLEMENTATION PLAN\*\*

## ### PHASE 1: PROJECT SETUP (Day 1)

## #### Step 1.1: Initialize Node.js Project

```
```bash
cd ~/vishruth/games/clusterrush
npm init -y
```
```

## #### Step 1.2: Install Dependencies

```
```bash
npm install three @types/three cannon-es @types/cannon-es serve
npm install -D typescript ts-node nodemon
```
```

## #### Step 1.3: Create Project Structure

```
```
clusterrush/
├── src/
│   ├── main.ts           # Entry point
│   ├── game.ts           # Game class
│   ├── player.ts         # Player logic
│   ├── physics.ts        # Physics engine
│   ├── input.ts          # Input handling
│   ├── ui.ts             # UI system
│   └── scene.ts           # Scene management
└──
```
```

```

├── config.ts # Game settings
├── assets/
│ ├── models/ # 3D models (GLTF)
│ ├── textures/ # Textures
│ └── sounds/ # Audio files
├── public/
│ └── index.html # Main HTML page
├── package.json
├── tsconfig.json
├── server.config.js # Node.js server config
└── README.md

```

```

```

### ### PHASE 2: CORE ENGINE (Day 2-3)

#### #### Step 2.1: Main Game Loop (src/main.ts)

```

```typescript
import { Game } from './game';

const game = new Game();
game.init();
game.start();
```

```

#### #### Step 2.2: Game Class (src/game.ts)

```

```typescript
import * as THREE from 'three';
import { Input } from './input';
import { Physics } from './physics';
import { UI } from './ui';
import { Player } from './player';

export class Game {
  private scene: THREE.Scene;
  private camera: THREE.PerspectiveCamera;
  private renderer: THREE.WebGLRenderer;
  private input: Input;
  private physics: Physics;
  private ui: UI;
  private player: Player;
  private clock: THREE.Clock;
  private isRunning: boolean;

  constructor() {
    this.scene = new THREE.Scene();
    this.camera = new THREE.PerspectiveCamera(75, window.innerWidth / window.innerHeight,
0.1, 1000);
    this.renderer = new THREE.WebGLRenderer({ antialias: true });
    this.input = new Input();
    this.physics = new Physics();
    this.ui = new UI();
    this.player = new Player();
    this.clock = new THREE.Clock();
    this.isRunning = false;
  }

  init(): void {
    this.setupRenderer();
    this.setupLights();
    this.setupCamera();
    this.setupEventListeners();
  }
}

```

```

    start(): void {
        this.isRunning = true;
        this.animate();
    }

    private setupRenderer(): void {
        this.renderer.setSize(window.innerWidth, window.innerHeight);
        this.renderer.setPixelRatio(window.devicePixelRatio);
        document.body.appendChild(this.renderer.domElement);
    }

    private setupLights(): void {
        const ambientLight = new THREE.AmbientLight(0xffffff, 0.6);
        this.scene.add(ambientLight);

        const directionalLight = new THREE.DirectionalLight(0xffffff, 1);
        directionalLight.position.set(10, 20, 10);
        this.scene.add(directionalLight);
    }

    private animate(): void {
        if (!this.isRunning) return;

        requestAnimationFrame(() => this.animate());

        const deltaTime = this.clock.getDelta();
        this.update(deltaTime);
        this.render();
    }

    private update(deltaTime: number): void {
        this.input.update();
        this.player.update(deltaTime);
        this.physics.update(deltaTime);
        this.ui.update();
    }

    private render(): void {
        this.renderer.render(this.scene, this.camera);
    }
}

```

PHASE 3: GAMEPLAY SYSTEMS (Day 4-6)

Step 3.1: Player System (src/player.ts)

```

``typescript
import * as THREE from 'three';
import { Input } from './input';

export class Player {
    private mesh: THREE.Group;
    private velocity: THREE.Vector3;
    private speed: number;
    private jumpForce: number;
    private isGrounded: boolean;

    constructor() {
        this.velocity = new THREE.Vector3(0, 0, 0);
        this.speed = 5;
        this.jumpForce = 10;
        this.isGrounded = true;
    }
}

```

```

    // Create player mesh
    this.mesh = new THREE.Group();

    const geometry = new THREE.BoxGeometry(1, 1, 1);
    const material = new THREE.MeshStandardMaterial({ color: 0x00ff00 });
    const cube = new THREE.Mesh(geometry, material);

    this.mesh.add(cube);
}

update(deltaTime: number): void {
    // Movement
    if (this.input.isPressed('w')) {
        this.velocity.z = -this.speed;
    }
    if (this.input.isPressed('s')) {
        this.velocity.z = this.speed;
    }
    if (this.input.isPressed('a')) {
        this.velocity.x = -this.speed;
    }
    if (this.input.isPressed('d')) {
        this.velocity.x = this.speed;
    }

    // Jump
    if (this.input.isPressed(' ') && this.isGrounded) {
        this.velocity.y = this.jumpForce;
        this.isGrounded = false;
    }

    // Apply physics
    this.velocity.y -= 9.81 * deltaTime; // Gravity
    this.mesh.position.add(this.velocity.clone().multiplyScalar(deltaTime));

    // Ground collision
    if (this.mesh.position.y <= 0) {
        this.mesh.position.y = 0;
        this.velocity.y = 0;
        this.isGrounded = true;
    }
}

getMesh(): THREE.Group {
    return this.mesh;
}
}

```

PHASE 4: LEVEL SYSTEM (Day 7-8)

Step 4.1: Level Manager

```
```typescript
```

```
import * as THREE from 'three';
```

```
export class LevelManager {
 private levelData: any[] = [];
 private currentLevel: number = 0;
```

```
 loadLevel(levelNumber: number): void {
 this.currentLevel = levelNumber;
```

```
 // Load level geometry
```

```

 const loader = new THREE.GLTFLoader();
 loader.load(`/assets/levels/level_${levelNumber}.gltf`, (gltf) => {
 scene.add(gltf.scene);
 });
}

getObstacles(): THREE.Mesh[] {
 // Return obstacle meshes for collision
 return [];
}
}
\..

```

---

### ### PHASE 5: PHYSICS & COLLISION (Day 9-10)

#### #### Step 5.1: Physics Engine

```

```typescript
import * as CANNON from 'cannon-es';

export class Physics {
    private world: CANNON.World;
    private bodies: CANNON.Body[] = [];

    constructor() {
        this.world = new CANNON.World();
        this.world.gravity.set(0, -9.82, 0);
    }

    update(deltaTime: number): void {
        this.world.step(1 / 60, deltaTime, 3);
    }

    addBody(body: CANNON.Body): void {
        this.world.addBody(body);
    }
}
\..

```

PHASE 6: UI SYSTEM (Day 11)

Step 6.1: HTML-based UI (no CORS issues!)

```

```typescript
export class UI {
 private scoreEl: HTMLElement;
 private levelEl: HTMLElement;
 private menuEl: HTMLElement;
 private hudEl: HTMLElement;

 constructor() {
 this.setupUIElements();
 }

 private setupUIElements(): void {
 // Create menu
 this.menuEl = document.createElement('div');
 this.menuEl.innerHTML = `
 <div class="menu-container">
 <h1>CLUSTER RUSH</h1>
 <button id="start-btn">START GAME</button>
 </div>
 `;
 }
}

```



```

 document.body.appendChild(this.menuEl);

 // Create HUD (hidden initially)
 this.hudEl = document.createElement('div');
 this.hudEl.style.display = 'none';
 this.hudEl.innerHTML = `
 <div class="hud-score">SCORE: 0</div>
 `;
 document.body.appendChild(this.hudEl);

 // Setup event listeners
 document.getElementById('start-btn')?.addEventListener('click', () => {
 this.menuEl.style.display = 'none';
 this.hudEl.style.display = 'block';
 });
 }
}

```

---

### ### PHASE 7: BUILD & TEST (Day 12-14)

#### #### Build Commands

```

```bash
npm run build    # Compile TypeScript
npm run serve    # Start local server on port 8080
```

```

#### #### Test Checklist

- [ ] Game loads in browser
- [ ] Menu appears
- [ ] Click START → HUD appears
- [ ] Player moves with WASD
- [ ] Jump with SPACE
- [ ] Score updates
- [ ] Cross-browser compatible

---

### ## 📅 \*\*IMPLEMENTATION TIMELINE\*\*

| Day   | Phase        | Deliverable                   |
|-------|--------------|-------------------------------|
| 1     | Setup        | Node.js project, dependencies |
| 2-3   | Core Engine  | Game loop, renderer           |
| 4-6   | Gameplay     | Player, input, UI             |
| 7-8   | Level System | Level loading                 |
| 9-10  | Physics      | Physics engine                |
| 11    | UI           | HTML-based UI                 |
| 12    | Build        | TypeScript build              |
| 13-14 | Testing      | Manual testing                |

**\*\*Total Time\*\*:** 14 days (2 weeks)

---

### ## ✅ \*\*SUCCESS CRITERIA\*\*

- [ ] Game loads in browser
- [ ] HTML UI displays correctly
- [ ] Game accepts keyboard inputs
- [ ] Player moves and jumps
- [ ] No CORS errors
- [ ] Cross-browser compatible

---

**\*\*Ready to start implementation when you give the go-ahead.\*\***

## # Project Status Enhanced

# Cluster Rush - Enhanced Project Status

## SPEC-DRIVEN + TEST-DRIVEN DEVELOPMENT APPROACH

##  ENHANCEMENTS COMPLETE

### What Was Added:

1. **\*\*Spec-Driven Development Framework\*\*** - `SPEC\_DRIVEN\_DEVELOPMENT.md`
  - Complete specification with user stories
  - Technical architecture definition
  - Success criteria and constraints
  - Decision points requiring user input
2. **\*\*Test-Driven Development Plan\*\*** - `TDD\_PLAN.md`
  - Strict RED→GREEN→REFACTOR enforcement
  - Weekly task breakdown with TDD cycles
  - Verification checkpoints and coverage targets
  - Risk mitigation strategies
3. **\*\*Enhanced Technical Direction\*\***
  - **\*\*Vite\*\*** instead of plain Node.js server (faster dev, better DX)
  - **\*\*Vitest\*\*** testing framework (optimized for Vite/browser code)
  - **\*\*TypeScript strict mode\*\*** (maximal type safety)
  - **\*\*Test pyramid structure\*\*** (unit/integration/E2E balance)

### What Remains for Implementation:

| File                         | Purpose                             | Status                                                                                                     |
|------------------------------|-------------------------------------|------------------------------------------------------------------------------------------------------------|
| -----                        | -----                               | -----                                                                                                      |
| `SPEC_DRIVEN_DEVELOPMENT.md` | <b>**Master specification**</b>     |  Complete             |
| `TDD_PLAN.md`                | <b>**TDD implementation guide**</b> |  Complete             |
| `REBUILD_PLAN.md`            | <b>**Original 14-day plan**</b>     |  Retained (reference) |
| `README.md`                  | <b>**Project overview**</b>         |  Needs update         |
| `PROJECT_STATUS.md`          | <b>**Current status**</b>           |  This file            |

---

##  ENHANCED APPROACH: SPEC + TDD

### Key Improvements Over Original Plan:

1. **\*\*Specification First\*\*** (not just planning)
  - Clear user stories with acceptance criteria
  - Technical constraints and success metrics
  - Open decisions flagged for user input
2. **\*\*Test-Driven Discipline\*\***
  - No production code without failing test first
  - Weekly coverage targets (80% → 85% → 90%)
  - Automation pyramid (unit/integration/E2E)
3. **\*\*Modern Tooling\*\***
  - Vite for faster builds and HMR
  - Vitest optimized for browser code
  - TypeScript strict mode from day 1
4. **\*\*Risk Management\*\***

- Identified technical risks with mitigations
- Verification checkpoints after each phase
- Performance targets and monitoring

---

### ### Technology Stack (Enhanced):

| Category       | Original          | Enhanced                            |
|----------------|-------------------|-------------------------------------|
| Build Tool     | Node.js + `serve` | <b>Vite</b> (faster dev, better DX) |
| Testing        | Manual testing    | <b>Vitest</b> + <b>Puppeteer</b>    |
| Type Safety    | Typescript        | <b>TypeScript strict mode</b>       |
| Dev Experience | Basic             | <b>Hot module replacement</b>       |

### ### Why This Enhancement:

| Original Limitation    | Enhanced Solution                               |
|------------------------|-------------------------------------------------|
| Vague success criteria | Quantifiable metrics (FPS, coverage, load time) |
| Ad-hoc testing         | TDD enforcement with coverage targets           |
| Reactive development   | Proactive spec-driven planning                  |
| Manual verification    | Automated validation checkpoints                |

---

## ## 📋 IMMEDIATE NEXT STEPS (TDD-ENFORCED)

### ### Day 1: Infrastructure with Tests First

- Initialize project with testing** (`npm init`, Vite config)
  - Red: Test that `npm run test` runs
  - Green: Configure Vitest with TypeScript
  - Verify: Empty test suite passes
- TypeScript strict mode**
  - Red: Test compilation fails without types
  - Green: Configure `tsconfig.json` strict options
  - Verify: `npm run build` succeeds
- Three.js rendering test**
  - Red: Test WebGL renderer creation fails
  - Green: Minimal Three.js renderer setup
  - Verify: Canvas appears, test passes

### ### Week 1 Focus: Core Systems (80% coverage target)

- Game loop with frame timing
- Input system with keyboard detection
- Basic scene management
- Physics system integration

### ### Week 2 Focus: Gameplay (85% coverage target)

- Player movement and jump mechanics
- Collision detection and scoring
- UI system (DOM-based, no CORS)
- Game state management

### ### Week 3 Focus: Polish & Optimization (90% coverage target)

- Performance optimization
- Cross-browser compatibility
- Final integration testing
- Documentation completion

---

## ## 🛠 TESTING STRATEGY

### ### Test Pyramid:

- **\*\*80% Unit Tests\*\***: Individual functions/systems
- **\*\*15% Integration Tests\*\***: System interactions
- **\*\*5% E2E Tests\*\***: Complete user flows

#### ### Coverage Requirements:

- **\*\*Phase 1 (Week 1)\*\***: 80%+ branch coverage
- **\*\*Phase 2 (Week 2)\*\***: 85%+ branch coverage
- **\*\*Phase 3 (Week 3)\*\***: 90%+ branch coverage

#### ### Critical Paths (100% coverage required):

1. Game initialization and cleanup
2. Input → Player movement chain
3. Collision detection → Game logic
4. State transitions (menu→play→gameOver)

---

## ## 📊 SUCCESS METRICS (Testable)

#### ### Technical Metrics:

- ✅ 90%+ test coverage (branch level)
- ✅ 60 FPS maintained under load
- ✅ <3 second initial load time
- ✅ No memory leaks in extended gameplay

#### ### User Experience Metrics:

- ✅ Input latency < 100ms
- ✅ All UI elements functional cross-browser
- ✅ No console errors/warnings
- ✅ Intuitive controls (WASD + SPACE)

#### ### Development Metrics:

- ✅ TDD compliance on all production code
- ✅ Daily test suite passes
- ✅ Weekly coverage reports
- ✅ No broken functionality in main branch

---

## ## 🔄 STATUS TRACKING

#### ### Current Status:

- **\*\*Specification\*\***: ✅ Complete (`SPEC\_DRIVEN\_DEVELOPMENT.md`)
- **\*\*TDD Plan\*\***: ✅ Complete (`TDD\_PLAN.md`)
- **\*\*Project Setup\*\***: ⌚ Ready to start
- **\*\*Implementation\*\***: ⌚ Not started

#### ### Blockers:

- None - Ready for implementation

#### ### Dependencies:

- User input on open questions in specification
- Development environment ready (Node.js installed)

---

## ## ✅ VERIFICATION CHECKLIST

#### ### Before Starting Implementation:

- [ ] User has reviewed and approved specification
- [ ] Open questions in spec have been addressed
- [ ] Development environment confirmed working
- [ ] TDD discipline agreed upon

#### ### After Each Task Completion:

- [ ] Tests written first (RED phase verified)
- [ ] Implementation passes tests (GREEN verified)
- [ ] All existing tests still pass (no regression)
- [ ] Coverage requirement met for phase
- [ ] Manual browser test confirms functionality

---

**\*\*Status\*\***:  Specification complete,  Ready for TDD implementation  
**\*\*Direction\*\***: Spec-driven + test-driven development  
**\*\*Next Action\*\***: Begin Day 1 TDD tasks in `TDD\_PLAN.md`  
**\*\*Confidence\*\***: High (structured approach with verification)

=====

# Readme Enhanced

# Cluster Rush - Enhanced README

## 🎮 3D Web Game with Spec-Driven + Test-Driven Development

## 📖 Overview

Cluster Rush is a 3D endless runner game being rebuilt from Godot WebGL to pure WebGL/Three.js to avoid CORS and rendering bugs. This project follows **\*\*spec-driven development\*\*** and **\*\*strict test-driven development\*\*** methodologies.

## 🚀 Quick Start

### Prerequisites

- Node.js v20.11.0 or higher
- Modern browser with WebGL support (Chrome 90+, Firefox 88+, Safari 14+)

### Installation & Development

```bash

Clone repository

git clone <repository-url>

cd clusterrush

Install dependencies

npm install

Start development server (with hot reload)

npm run dev

Run tests (TDD workflow)

npm run test

Build for production

npm run build

Preview production build

npm run preview

```

**\*\*Development Server\*\***: http://localhost:5173

## 📁 Project Structure

...

clusterrush/

├─ src/	# Source code
├─ tests/	# All test files (unit/integration/e2e)
├─ public/	# Static assets and HTML
├─ config/	# Build and tool configurations
└─ docs/	# Documentation

```
└─ *.md
...

```

## # Planning and specification files

### ### Key Documentation Files

File	Purpose
`SPEC_DRIVEN_DEVELOPMENT.md`	Complete specification with user stories
`TDD_PLAN.md`	Test-driven development implementation guide
`REBUILD_PLAN.md`	Original 14-day implementation plan
`PROJECT_STATUS_ENHANCED.md`	Current project status and next steps

## ## 🛠️ Testing Strategy (TDD Enforcement)

### ### Test Commands

```
```bash
npm run test          # Run all tests
npm run test:unit      # Unit tests only
npm run test:integration # Integration tests
npm run test:e2e       # Browser E2E tests (Puppeteer)
npm run test:watch     # Watch mode for TDD workflow
npm run test:coverage  # Generate coverage report
```

```

### ### Coverage Requirements

- **\*\*Week 1\*\***: 80%+ branch coverage
- **\*\*Week 2\*\***: 85%+ branch coverage
- **\*\*Week 3\*\***: 90%+ branch coverage

### ### TDD Workflow

1. **\*\*RED\*\***: Write failing test describing expected behavior
2. **\*\*Verify RED\*\***: Confirm test fails (not error)
3. **\*\*GREEN\*\***: Write minimal code to pass test
4. **\*\*Verify GREEN\*\***: Confirm test passes
5. **\*\*REFACTOR\*\***: Clean code without changing behavior
6. **\*\*Verify REFACTOR\*\***: All tests still pass

**\*\*Rule\*\***: No production code without a failing test first.

## ## 🎯 Success Criteria

### ### Technical Metrics

- ✅ 90%+ test coverage (branch level)
- ✅ 60 FPS maintained under load
- ✅ <3 second initial load time
- ✅ No memory leaks in extended gameplay

### ### User Experience Metrics

- ✅ Input latency < 100ms
- ✅ All UI elements functional cross-browser
- ✅ No console errors/warnings
- ✅ Intuitive controls (WASD + SPACE)

## ## 🛠️ Technology Stack

### ### Core Dependencies

- **\*\*Three.js v0.162.0\*\*** - 3D rendering engine
- **\*\*Cannon-es v0.20.0\*\*** - Physics simulation
- **\*\*TypeScript v5.4.5\*\*** - Type-safe development

### ### Development Tools

- **\*\*Vite v5.3.0\*\*** - Fast builds and dev server
- **\*\*Vitest v1.6.0\*\*** - Testing framework
- **\*\*Puppeteer v22.6.0\*\*** - Browser automation

## ## 📖 Development Phases

### ### Phase 1: Core Infrastructure (Week 1)

- Project setup with TDD from day 1
- Core game loop and input system
- Basic Three.js rendering pipeline
- **\*\*Target\*\***: 80% test coverage

### ### Phase 2: Gameplay Systems (Week 2)

- Player movement and jump mechanics
- Physics integration with collision detection
- Scoring system and game state management
- **\*\*Target\*\***: 85% test coverage

### ### Phase 3: Polish & Optimization (Week 3)

- UI system with DOM integration (no CORS!)
- Performance optimization and cross-browser testing
- Final integration and documentation
- **\*\*Target\*\***: 90% test coverage

## ## 🚨 Important Notes

### ### Why Not Godot WebGL?

- **\*\*WebGL Bug\*\***: Control UI nodes don't render in Godot 4.x WebGL
- **\*\*CORS Barrier\*\***: Can't communicate between HTML overlay and Godot engine
- **\*\*Frozen State\*\***: Game engine loads but doesn't accept inputs
- **\*\*Solution\*\***: Pure JavaScript/TypeScript with Three.js avoids all these issues

### ### Key Architecture Decisions

1. **\*\*DOM-based UI\*\***: HTML overlay instead of Canvas UI (no CORS issues)
2. **\*\*Single-page app\*\***: No iframe isolation, direct DOM access
3. **\*\*Test-first approach\*\***: Every feature developed with failing test first
4. **\*\*TypeScript strict mode\*\***: Maximum type safety from the beginning

## ## 🤝 Contributing

### ### Development Workflow

1. Always start with a failing test (RED)
2. Write minimal implementation (GREEN)
3. Clean up code (REFACTOR)
4. Verify all tests pass before commit
5. Check coverage meets phase requirements

### ### Code Standards

- TypeScript strict mode compliance
- JSDoc comments for public APIs
- Consistent naming conventions
- No console errors/warnings in production

## ## 📄 License

MIT License - see LICENSE file for details

## ## 🧑🏫 Getting Help

- Review `SPEC\_DRIVEN\_DEVELOPMENT.md` for detailed specifications
- Check `TDD\_PLAN.md` for implementation guidance
- Run tests after any changes

---

**\*\*Project Status\*\***: ✅ Planning complete, ⏳ Ready for TDD implementation

**\*\*Development Approach\*\***: Spec-driven + Test-driven development

**\*\*Expected Timeline\*\***: 3 weeks (14 working days)

**\*\*Confidence Level\*\***: High (structured approach with verification checkpoints)

=====

## # Cluster Rush E2E Test Report

## # Cluster Rush Game - Comprehensive E2E Browser Test Report

## ## Executive Summary

The Cluster Rush Godot WebGL game loads successfully but is **completely non-responsive** to user interactions. Despite extensive testing with mouse clicks, keyboard inputs, and various interaction patterns, the game shows no visual response or screen transitions. The game appears "stuck" on the initial loading screen.

## ## Test Details

## ### Test Environment

- **Game**: Cluster Rush (Godot WebGL)
- **Location**: `/home/srinivasvkumar/vishruth/games/clusterrush/`
- **Server**: Python webgl\_server.py running on port 8080
- **Browser**: Automated testing via browser-use CLI
- **Test Date**: Current session

## ### File Structure Analysis

```
Game Directory: /home/srinivasvkumar/vishruth/games/clusterrush/
├── Builds/WebGL/
│ ├── index.html (5.4KB) # Main HTML file
│ ├── index.js (279KB) # Godot JavaScript runtime
│ ├── index.wasm (39.5MB) # WebAssembly game binary
│ ├── index.pck (28.5MB) # Game assets package
│ └── 16 other supporting files
├── webgl_server.py (3KB) # HTTP server script
└── 60 other project files
```

## ### Initial Game State (Confirmed)

- ✓ **Game loads**: Canvas appears at 1280x633 pixels
- ✓ **Display normal**: No `display:none` or `visibility:hidden`
- ✓ **Page title**: "Cluster Rush" (initially, though later blank)
- ✓ **Viewport matches canvas**: 1280x633 pixels
- ✓ **Server running**: Port 8080 accessible
- ✗ **Interactive elements**: None visible

## ## Interaction Testing Results

## ### Mouse Click Tests

| Test Type     | Locations Tested             | Result        |
|---------------|------------------------------|---------------|
| Single clicks | Center, all 4 corners, edges | ✗ No response |
| Double clicks | Center position              | ✗ No response |
| Long press    | 1-second hold at center      | ✗ No response |
| Rapid clicks  | 10 positions across canvas   | ✗ No response |
| Extreme edges | Very top/bottom/left/right   | ✗ No response |

## ### Keyboard Tests

| Keys Tested          | Purpose                  | Result        |
|----------------------|--------------------------|---------------|
| Space, Enter, Escape | Menu/start buttons       | ✗ No response |
| WASD keys            | Movement controls        | ✗ No response |
| Arrow keys           | Directional controls     | ✗ No response |
| All key events       | Keydown/keyup simulation | ✗ No response |

## ### Advanced Interaction Patterns

- ✗ **Sequential clicking**: Multiple positions in sequence - ✗ No response
- ✗ **Pattern clicking**: Geometric patterns on canvas - ✗ No response
- ✗ **Focus switching**: Canvas focus/blur events - ✗ No effect
- ✗ **Viewport interaction**: Clicking outside canvas - ✗ No response



## ## Screen Changes & Visual Feedback

- **\*\*Screen transitions\*\***: ❌ None observed
- **\*\*UI elements appearing\*\***: ❌ None appeared
- **\*\*Visual feedback\*\***: ❌ No changes to canvas
- **\*\*Loading indicators\*\***: ❌ None visible
- **\*\*Error messages\*\***: ❌ None displayed (browser console not accessible)

## ## Game Responsiveness Assessment

| Metric                                                                       | Score | Notes                       |
|------------------------------------------------------------------------------|-------|-----------------------------|
| Mouse responsiveness                                                         | 0/10  | No response to any clicks   |
| Keyboard responsiveness                                                      | 0/10  | No response to any keys     |
| Visual feedback                                                              | 0/10  | Canvas remains static       |
| Error reporting                                                              | 0/10  | No visible error indicators |
| <b>**Overall responsiveness**   **0/10**   **Completely non-responsive**</b> |       |                             |

## ## Identified Issues & Potential Causes

### ### Primary Issue

The game loads a canvas but never progresses beyond the initial static screen, suggesting the Godot engine may not have fully initialized or is stuck in a loading state.

### ### Potential Root Causes (Ranked by Likelihood)

- \*\*WebGL Context Failure\*\*** (Most Likely)
  - Godot WebGL runtime may have failed silently
  - Browser/WebGL compatibility issues
  - Missing WebGL extensions or capabilities
- \*\*Asset Loading Failures\*\***
  - Large files (39MB WASM, 28MB PCK) may not be loading completely
  - Network/timeout issues during asset loading
  - Corrupted game files
- \*\*Godot Initialization Errors\*\***
  - JavaScript runtime errors preventing game start
  - Missing dependencies or initialization sequence failures
  - Godot version compatibility issues
- \*\*Input System Not Initialized\*\***
  - Game may require specific initialization before accepting input
  - Input handlers not attached to canvas
  - Event listener registration failures
- \*\*Game Logic Bugs\*\***
  - Infinite loop or stuck state in game code
  - Scene transition failures
  - Menu system not implemented/activated

## ## Technical Observations

### ### File Analysis Findings

- ✅ Game files exist and are properly structured
- ✅ Server correctly configured with CORS headers
- ✅ Files are large but typical for Godot WebGL exports
- ? JavaScript/WebAssembly files may have loading/execution issues

### ### Browser Console Limitations

Unable to access browser console during automated testing, which limits error diagnosis. Critical WebGL or JavaScript errors would be visible in console.

### ### Canvas Behavior Notes

- Canvas initially visible at correct dimensions (1280x633)
- Canvas disappears after extensive interaction testing (potential crash)

- No visual changes under any test conditions

## ## Recommendations for Debugging

### ### Immediate Actions

1. **Check browser developer console** for WebGL/JavaScript errors
2. **Monitor network tab** during loading to verify all assets download
3. **Test in different browsers** (Chrome, Firefox) to isolate WebGL issues

### ### Code/Configuration Checks

4. **Verify Godot export settings** for WebGL compatibility
5. **Add debug logging** to game startup sequence
6. **Test game in Godot editor** vs WebGL export to isolate issues

### ### Server/Deployment Fixes

7. **Check server headers** are correctly set for WebGL
8. **Verify file permissions** and MIME types
9. **Test with simpler Godot WebGL project** as baseline

### ### Advanced Debugging

10. **Enable Godot debug mode** in WebGL export
11. **Check memory usage** during loading (large files may cause OOM)
12. **Examine Godot JavaScript console** for engine-specific errors

## ## Test Metrics & Coverage

- **Total interaction attempts**: ~30+ input events
- **Test duration**: Several minutes of active testing
- **Input methods tested**: 2 (mouse, keyboard)
- **Interaction patterns**: 5+ different patterns
- **Screen areas covered**: Entire canvas + viewport edges
- **Visual monitoring**: Continuous observation for changes

## ## Conclusion

The Cluster Rush WebGL game has a **critical initialization issue** preventing user interaction. While the game loads and displays a canvas, the core game logic appears frozen or non-functional. The lack of response to any input type (mouse or keyboard) suggests either:

1. The Godot engine failed to initialize properly
2. WebGL context creation failed silently
3. Game assets are not loading completely
4. A fatal error occurs during startup

**Immediate priority**: Access browser developer console to identify specific WebGL or JavaScript errors that are preventing game execution.

**Confidence in findings**: High - extensive testing with multiple interaction patterns consistently shows zero responsiveness.

---

\*Report generated by automated browser testing agent\*

=====

## ## Complete Skills Reference for Game Development

### ### Software Engineering & Implementation

- **implement**: Implement work from spec or tickets
- **subagent-driven-development**: Execute plans via `delegate_task` subagents
- **source-driven-development**: Ground decisions in official documentation
- **code-simplification**: Refactoring for clarity
- **systematic-debugging**: 4-phase root cause debugging

- **debugging-and-error-recovery**: When tests fail or errors occur

### ### Planning & Task Breakdown

- **planning-and-task-breakdown**: Breaking work into ordered tasks
- **planning-with-files**: Persistent file-based planning
- **spec-driven-development**: Creating specs before coding
- **writing-plans**: Implementation plans with bite-sized tasks
- **wayfinder**: Planning huge chunks of work
- **domain-modeling**: Pinning down domain terminology
- **grill-me**: Relentless interview to sharpen plans
- **idea-refine**: Refining raw ideas

### ### Testing & Quality Assurance

- **test-driven-development**: TDD with RED-GREEN-REFACTOR
- **browser-testing-with-devtools**: Testing in real browsers via Chrome DevTools
- **playwright**: Real browser automation from terminal
- **webapp-testing**: Testing local web applications with Playwright
- **verify-and-stop**: Proving work meets acceptance conditions

### ### Browser Automation & Web Testing

- **agent-browser**: Browser automation CLI for AI agents
- **playwright**: Real browser automation
- **browser-debugging**: Debug client-side issues

### ### Delegation & Subagent Orchestration

- **subagent-driven-development**: Execute plans via subagents
- **delegate-setup**: Configure delegation fleet lanes
- **kanban-orchestrator**: Decomposition playbook
- **kanban-worker**: Kanban worker pitfalls

### **Delegate Implementers Available**:

- **aider-delegate**, **codex-delegate**, **claude-delegate**, **cline-delegate**
- **copilot-delegate**, **cursor-delegate**, **grok-delegate**, **kimidelegate**
- **qoder-delegate**, **vibe-delegate**, **warp-delegate**, **zcode-delegate**
- **opencode-delegate**, **pi-delegate**, **commandcode-delegate**, **omp-delegate**, **agy-delegate**

### ### Game Development Specific

- **godot-webgl-development**: Godot 4.x web games
- **responsive-html5-canvas**: Responsive HTML5 Canvas games
- **frontend-ui-engineering**: Production-quality UIs
- **frontend-design**: Visual design guidance
- **api-and-interface-design**: API and interface design

### ### Code Management & GitHub

- **github**: GitHub via gh CLI
- **github-issues**: Create, triage, label issues
- **github-pr-workflow**: PR lifecycle management
- **github-code-review**: Code review
- **caveman-commit**: Ultra-compressed commit messages

### ### Research & Documentation

- **research**: High-trust primary sources
- **arxiv**: Search arXiv papers
- **llm-wiki**: Karpathy's LLM Wiki
- **pdf**: Reading, creating, editing PDFs

### ### MCP & Tool Integration

- **native-mcp**: MCP client configuration
- **mcp-builder**: Creating MCP servers
- **hermes-agent**: Configure Hermes Agent

---

## ## Recommended Workflow for Cluster Rush

### ### Multi-Agent Orchestration

\\\

```

Boss Bot (Project Manager/Orchestrator)
├── Game Dev Agent (Implementation)
├── Game Test Agent (Browser Testing)
├── Researcher Agent (Documentation)
└── Reviewer Agent (Code Quality)

```

...

### ### Kanban Board Configuration

- Board: `cluster-rush` (separate from default/sowmya)
- max\_in\_progress: 3
- max\_in\_progress\_per\_profile: 1
- failure\_limit: 2
- dispatch\_in\_gateway: true
- orchestrator\_profile: orchestrator
- default\_assignee: orchestrator

### ### Implementation Workflow

1. **Planning Phase** (Complete) - spec-driven-development, writing-plans
2. **Implementation Phase** - subagent-driven-development with TDD
3. **Testing Phase** - playwright, webapp-testing, browser-testing-with-devtools
4. **Debugging** (As Needed) - systematic-debugging, browser-debugging

---

## ## Next Steps for Cluster Rush

### ### Immediate Actions (Day 1)

1. **Task 1.1**: Initialize project with testing infrastructure
  - Write failing test for `npm run test`
  - Configure Vitest with TypeScript
  - Verify: `npm test` → "✓ 0 passed"
2. **Task 1.2**: TypeScript strict mode
  - Write test for compilation
  - Configure tsconfig.json with strict options
  - Verify: `npm run build` compiles without errors
3. **Task 1.3**: Three.js rendering test
  - Write failing test for WebGL renderer
  - Implement minimal renderer
  - Verify: Canvas appears in browser

### ### Week 1 Focus: Core Infrastructure (80% coverage)

- Game loop with frame timing
- Input system with keyboard detection
- Basic scene management
- Physics system integration

### ### Week 2 Focus: Gameplay (85% coverage)

- Player movement and jump mechanics
- Collision detection and scoring
- UI system (DOM-based, no CORS)
- Game state management

### ### Week 3 Focus: Polish & Optimization (90% coverage)

- Performance optimization
- Cross-browser compatibility
- Final integration testing
- Documentation completion

---

## ## Telegram Bot Configuration

**\*\*Available Bots\*\*:**

- boss\_bot: @Srini\_Boss\_bot
- default: @Srinivasvkumar\_bot
- game-dev: @Srini\_gamedev\_bot
- game-tester: @Srini\_Game\_tester\_bot
- hr\_bot: @Srini\_Hr\_bot
- implementer: @Srini\_implementer\_bot
- orchestrator: @Srini\_Orchestrator\_bot
- researcher: @Srini\_researcher\_bot
- reviewer: @Srini\_Reviewer\_bot
- sowmya: @ca\_sowmya\_n\_bot

**\*\*User ID Mapping\*\*:**

- srinivas (default profile): 8416650003
- sowmya: 8629486921

---

**\*\*Document Generated\*\*:** 2026-09-08

**\*\*Total Sections\*\*:** Complete analysis with skills reference

**\*\*Status\*\*:** Ready for implementation